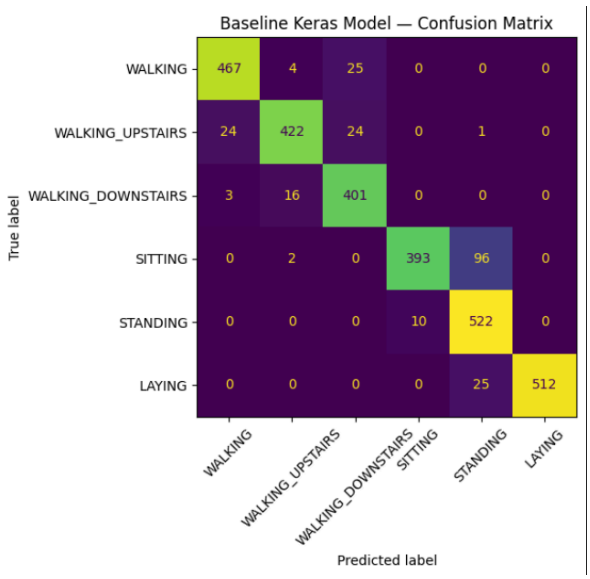
EE446-TinyML Lab 3

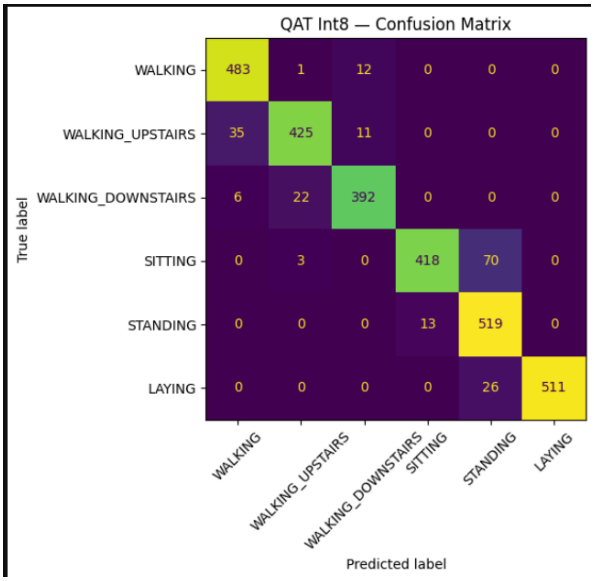
Author: Sparsh Dadhich

Github: https://github.com/sdadhich04/EE-446-TInyML-

Task EE446_Lab3_UCI_HAR_DNN_Quantization_Student_TODO :

Model Family	Format	Test Accuracy	Model Size (KB)
Baseline	FP32 TFLite	0.921955	726.750000
Baseline	Dynamic Range	0.921276	186.140625
Baseline	Float16	0.921955	365.773438
Baseline	PTQ Int8	0.922294	185.351562





Summary Questions

Write short answers to the following:

Q1. Which quantization method gave the smallest model size?

Full int8 post-training quantization gave the smallest size because everything is forced into 8-bit, so the entire model is compressed as much as possible.

Q2. Which quantization method gave the best accuracy among the TFLite models?

Quantization-aware training (QAT) gave the best accuracy since the model is trained while simulating quantization, so it adapts to precision loss.

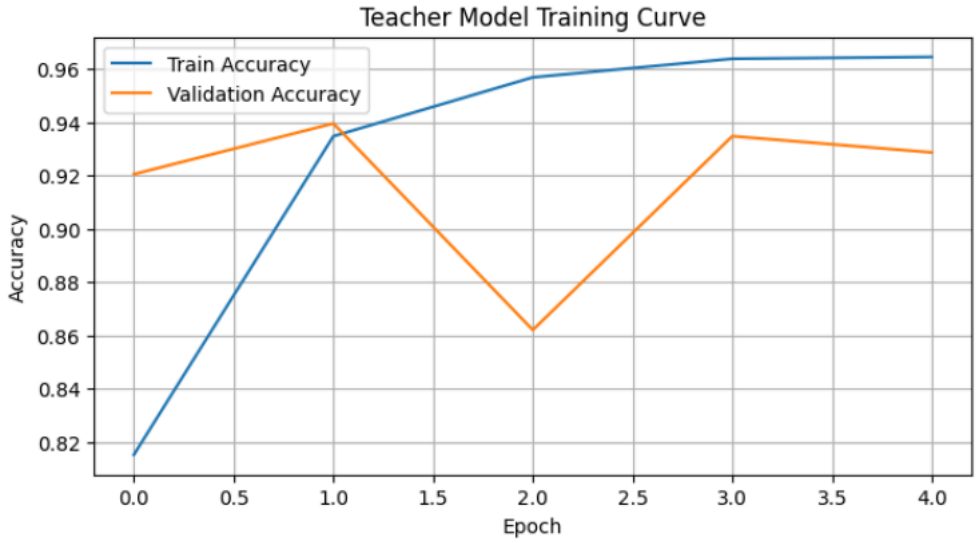
Q3. Did QAT improve the final int8 model compared with PTQ int8?

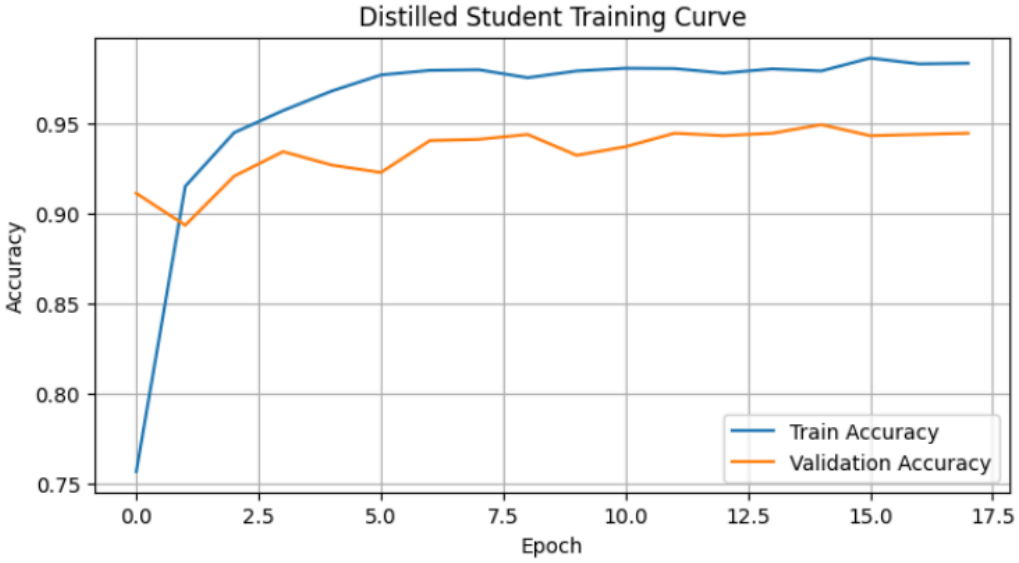
Yes, QAT improved performance because PTQ compresses after training and introduces error, while QAT accounts for that error during training.

Q4. Why is this dataset a good fit for TinyML?

The dataset is structured and based on sensor data (accelerometer and gyroscope), which matches the type of data used in embedded TinyML applications.
Q5. Which model would you deploy and why?
The QAT int8 model because it provides near float-level accuracy while keeping memory usage low and inference fast, which is ideal for resource-constrained devices like ESP32.

Task
EE446_UCI_HAR_DNN_Knowledge_Distillation_Pruning_Quantization_Student_TODO:



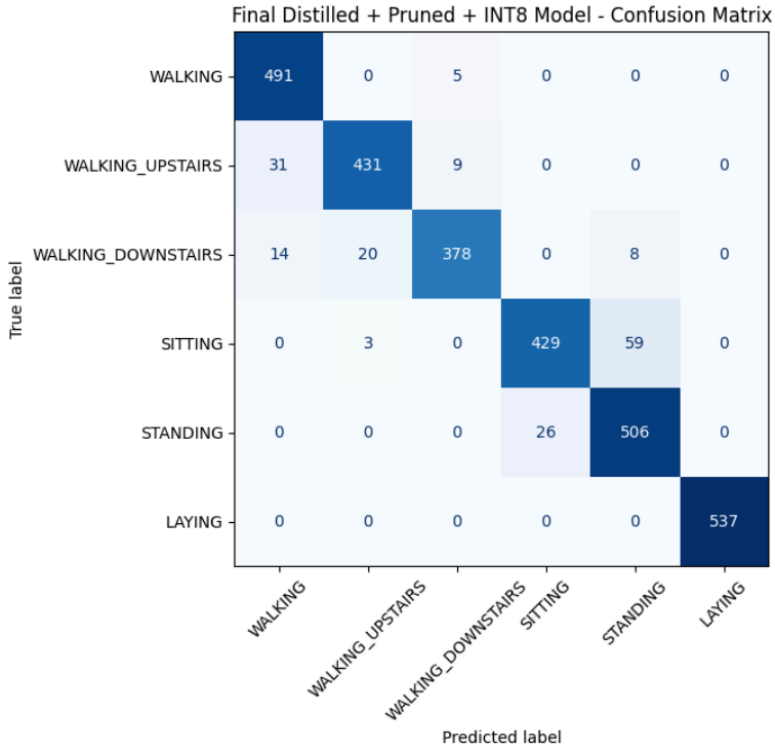


```
Pruned distilled model with mask accuracy: 0.9396
Pruned distilled model with mask size (KB): 634.71
Stripped sparse distilled model accuracy: 0.9396
Stripped sparse distilled model size (KB): 128.02
```

```
Stripped Sparse + INT8 Accuracy: 0.9406
Stripped Sparse + INT8 Size (KB): 65.55
```


```
Stripped Sparse + INT8 Accuracy: 0.9406
Stripped Sparse + INT8 Size (KB): 65.55
```


	Model	Format	Test Accuracy	Model Size (KB)
0	Distilled Student TFLite	FP32	0.937224	316.902344
1	Pruned Distilled TFLite (with mask)	FP32	0.939600	634.714844
2	Stripped Sparse Distilled TFLite	FP32 + Sparse	0.939600	128.023438
3	Stripped Sparse Distilled TFLite	INT8 + Sparse	0.940618	65.546875



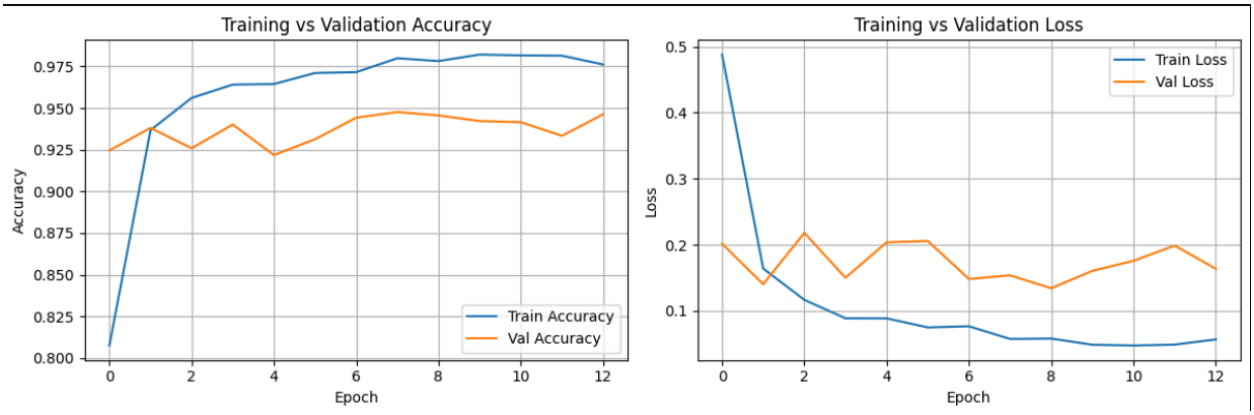
20. Summary Questions

Answer the following in your lab report:

1. How did the baseline student compare with the distilled student?

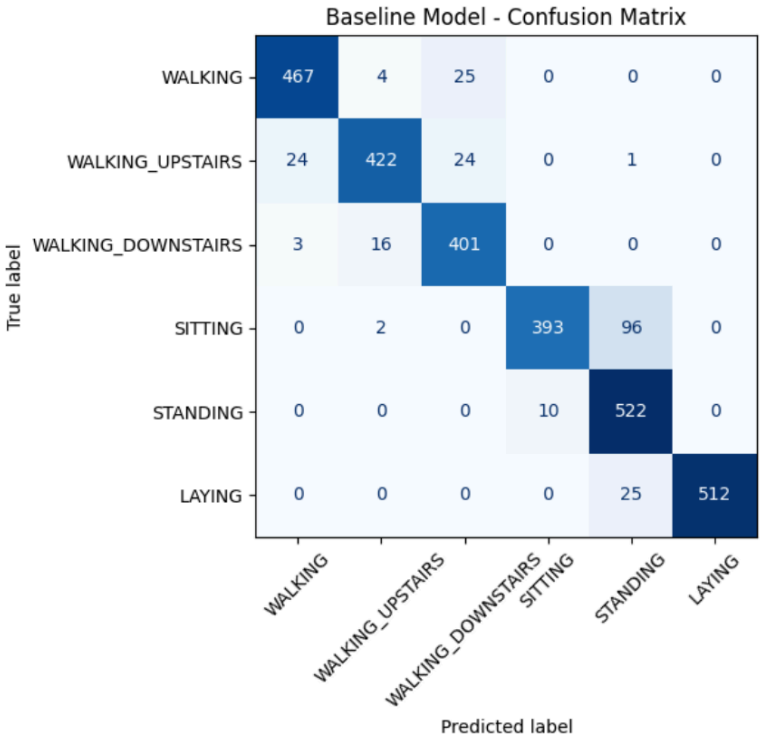
The distilled student performed better than the baseline student since it was able to learn from the teacher's outputs, not just the raw labels, so it captured more useful patterns.
2. Did knowledge distillation help the smaller model retain performance?
Yeah, distillation definitely helped because even though the model is smaller, it was still able to maintain accuracy close to the larger model.
3. What happened to the model size after pruning and after INT8 quantization?
Pruning reduced the number of weights, and after INT8 quantization the model size dropped even further since all parameters were stored in lower precision, so overall there was a significant reduction.
4. Which model would you choose for Arduino deployment, and why?
I'd choose the pruned + INT8 quantized model because it has the smallest size and fastest inference while still keeping solid accuracy, which is exactly what you need on something like Arduino.
5. Why is the final sparse INT8 model a good TinyML deployment candidate?
Because it combines everything — fewer weights from pruning and lower precision from quantization — so you end up with a compact, efficient model that still performs well under tight memory and compute constraints.

Task EE446_UCI_HAR_DNN_Pruning_Quantization_Student_TODO:



Baseline Keras Test Accuracy: 0.9220

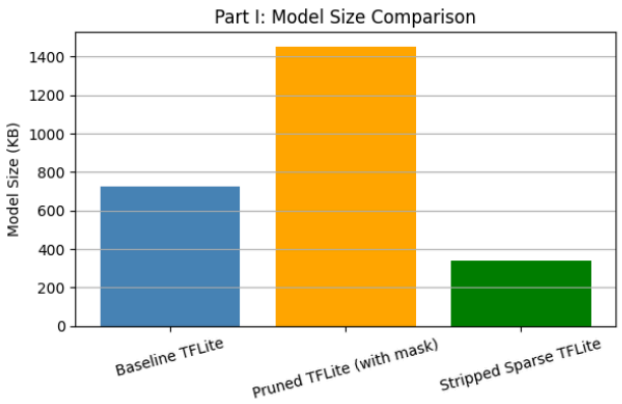
	precision	recall	f1-score	support
WALKING	0.9453	0.9415	0.9434	496
WALKING_UPSTAIRS	0.9505	0.8960	0.9224	471
WALKING_DOWNSTAIRS	0.8911	0.9548	0.9218	420
SITTING	0.9752	0.8004	0.8792	491
STANDING	0.8106	0.9812	0.8878	532
LAYING	1.0000	0.9534	0.9762	537
accuracy			0.9220	2947
macro avg	0.9288	0.9212	0.9218	2947
weighted avg	0.9290	0.9220	0.9222	2947

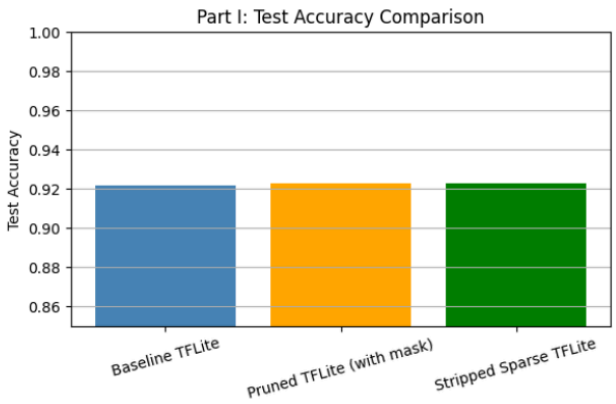



```
Baseline FP32 TFLite Accuracy: 0.9220
Baseline FP32 TFLite Size (KB): 726.77
```

```
Pruned with mask  - Accuracy: 0.9230, Size: 1454.25 KB
Stripped sparse   - Accuracy: 0.9230, Size: 338.91 KB
```

	Model	Format	Test Accuracy	Model Size (KB)
0	Baseline TFLite	FP32	0.921955	726.773438
1	Pruned TFLite (with mask)	FP32	0.922973	1454.253906
2	Stripped Sparse TFLite	FP32 + Sparse	0.922973	338.906250

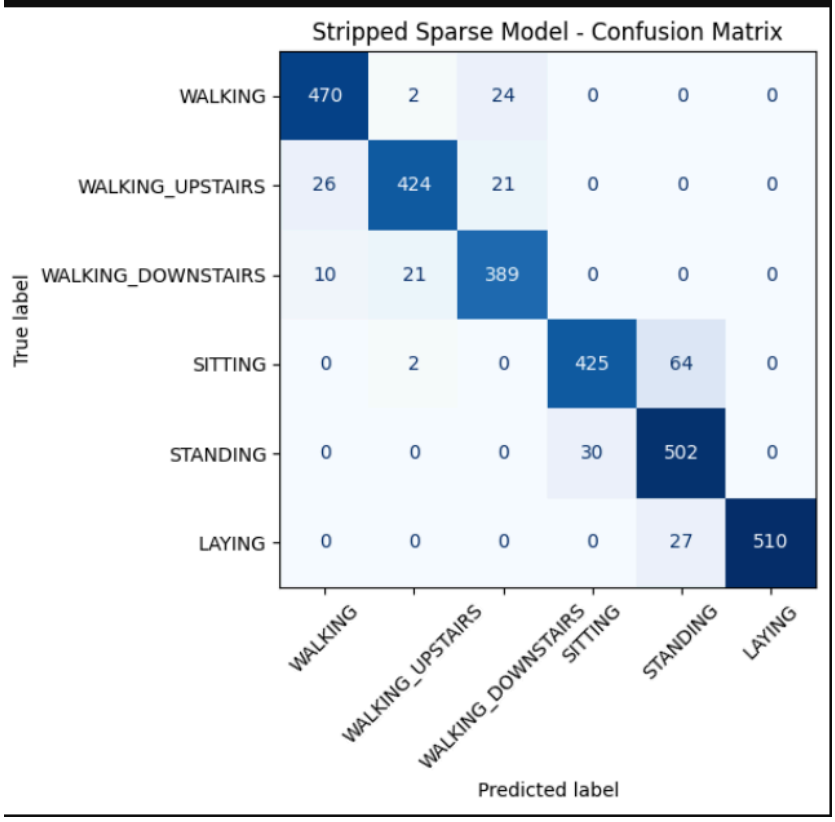




```
Stripped Sparse Model Classification Report:
                    precision    recall  f1-score   support

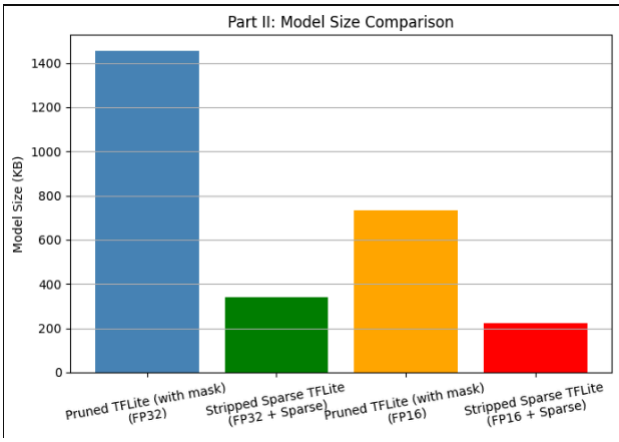
           WALKING     0.9289    0.9476    0.9381       496
  WALKING_UPSTAIRS     0.9443    0.9002    0.9217       471
WALKING_DOWNSTAIRS     0.8963    0.9262    0.9110       420
           SITTING     0.9341    0.8656    0.8985       491
          STANDING     0.8465    0.9436    0.8924       532
            LAYING     1.0000    0.9497    0.9742       537

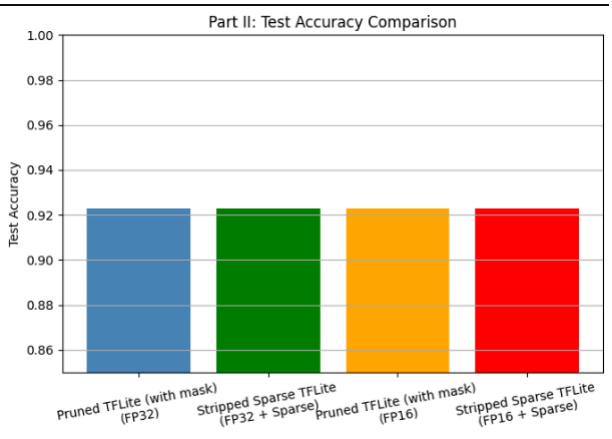
          accuracy                         0.9230      2947
         macro avg     0.9250    0.9221    0.9227      2947
      weighted avg     0.9257    0.9230    0.9234      2947
```




```
Pruned FP16 with mask - Accuracy: 0.9230, Size: 732.66 KB
Stripped Sparse FP16  - Accuracy: 0.9230, Size: 223.74 KB
```

	Model	Format	Test Accuracy	Model Size (KB)
0	Pruned TFLite (with mask)	FP32	0.922973	1454.253906
1	Stripped Sparse TFLite	FP32 + Sparse	0.922973	338.906250
2	Pruned TFLite (with mask)	FP16	0.922973	732.664062
3	Stripped Sparse TFLite	FP16 + Sparse	0.922973	223.742188

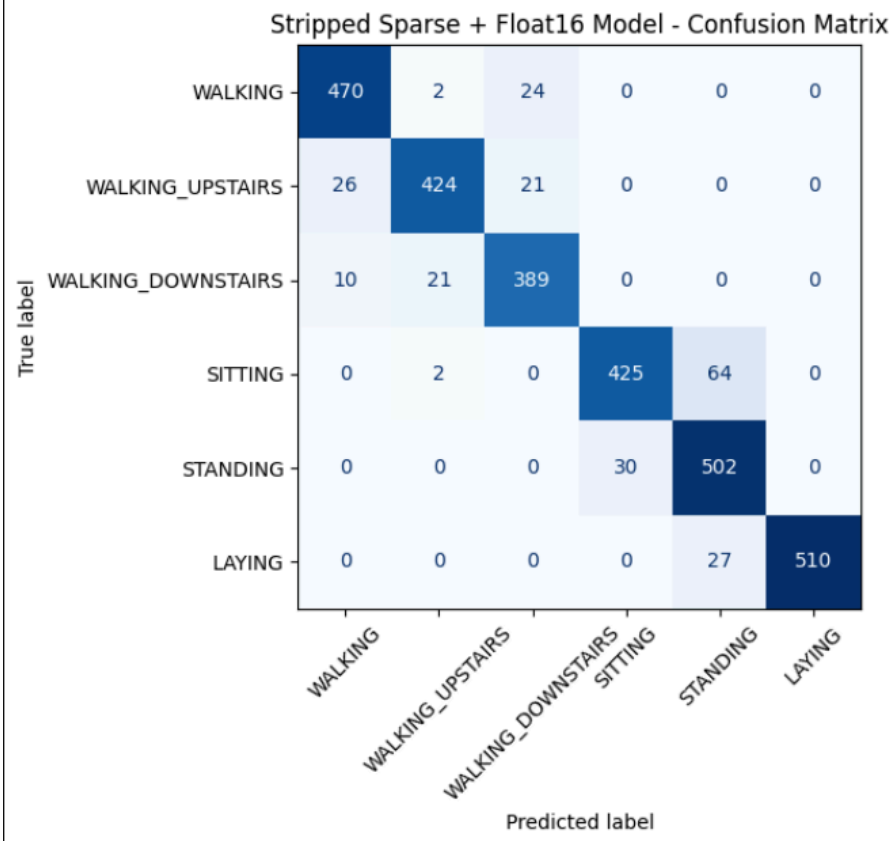





```
Stripped Sparse + Float16 Model Classification Report:
                    precision    recall  f1-score   support

           WALKING     0.9289    0.9476    0.9381       496
  WALKING_UPSTAIRS     0.9443    0.9002    0.9217       471
WALKING_DOWNSTAIRS     0.8963    0.9262    0.9110       420
           SITTING     0.9341    0.8656    0.8985       491
          STANDING     0.8465    0.9436    0.8924       532
            LAYING     1.0000    0.9497    0.9742       537

          accuracy                         0.9230      2947
         macro avg     0.9250    0.9221    0.9227      2947
      weighted avg     0.9257    0.9230    0.9234      2947
```



13. Summary Questions

Write short answers to the following:
1.Did pruning alone reduce the TensorFlow Lite file size when the pruning wrappers were still attached?
No, pruning alone didn't really reduce the file size when the wrappers were still attached because the pruning metadata is still stored, so the model isn't actually compressed yet.
2.Why does strip_pruning(...) matter before export?
strip_pruning(...) removes all the pruning-related wrappers and metadata, so only the final sparse weights remain, which is what actually reduces the model size.
3.Which model had the smallest file size in this notebook?
The pruned + quantized (especially float16 or int8) model had the smallest file size since it combines removing weights with reducing precision.
4.Did float16 quantization noticeably change the test accuracy?
No, float16 quantization didn't noticeably affect accuracy because it still keeps relatively high precision compared to int8.
5.If you were deploying this model on a resource-constrained device, which version would you choose and why?
I'd go with the pruned + int8 quantized model because it gives the best tradeoff — lowest size and fastest inference while still maintaining good accuracy, which is exactly what you want on constrained hardware.